

Lab Task 1: Bookstore Inventory System

Scenario:

You are designing an inventory system for a bookstore that stores book records by ISBN. Fast lookup and range queries are required for stock audits and price checks.

Task Requirements:

Each node should contain:

- ISBN (int, unique key)
- Title (string)
- Author (string)
- Price (float)

Core Functionalities:

- addBook(int isbn, string title, string author, float price)
- findBook(int isbn)
- removeBook(int isbn)
- displayBooksInOrder()
- getInorderISBNs() for tests

Advanced Operations:

- getBooksInRange(int low, int high) -> vector<Book>
- getTotalBooks()
- getHeight()

Note: Duplicate ISBNs should be rejected

Google Test Cases:

Note: Some testcases have vectors in them. You need to replace them with an appropriate list or array

```
TEST(BookBSTTest, AddAndFindBook) {  
  
    BookBST bst;  
  
    bst.addBook(123, "DSA", "Mark", 35.5);
```

```

    bst.addBook(456, "AI", "Nora", 50.0);

    Book b = bst.findBook(456);

    EXPECT_EQ(b.Title, "AI"); // Expected correct book details

    EXPECT_EQ(b.Author, "Nora");

}

TEST(BookBSTTest, FindNonExistingBook) {

    BookBST bst;

    bst.addBook(123, "DSA", "Mark", 35.5);

    Book b = bst.findBook(999);

    EXPECT_EQ(b.Title, ""); // Expected empty book when not found

}

TEST(BookBSTTest, DeleteLeafBook) {

    BookBST bst;

    bst.addBook(50, "A", "X", 20);

    bst.addBook(30, "B", "Y", 15);

    bst.addBook(70, "C", "Z", 30);

    bst.removeBook(30);

    EXPECT_EQ(bst.findBook(30).Title, ""); // Expected removed

}

TEST(BookBSTTest, DeleteBookWithOneChild) {

```

```
BookBST bst;

bst.addBook(100, "Root", "Auth", 40);

bst.addBook(120, "Child", "Auth2", 45);

bst.removeBook(100);

EXPECT_EQ(bst.findBook(100).Title, ""); // Expected promotion of child

EXPECT_EQ(bst.findBook(120).Title, "Child");

}

TEST(BookBSTTest, DeleteBookWithTwoChildren) {

    BookBST bst;

    bst.addBook(50, "R", "A", 10);

    bst.addBook(30, "L", "B", 15);

    bst.addBook(70, "M", "C", 20);

    bst.addBook(60, "LM", "D", 12);

    bst.removeBook(50);

    EXPECT_EQ(bst.findBook(50).Title, ""); // Root replaced by successor

}

TEST(BookBSTTest, BooksInRange) {

    BookBST bst;

    bst.addBook(10, "A", "X", 20);

    bst.addBook(20, "B", "Y", 30);

    bst.addBook(30, "C", "Z", 40);
```

```

    vectr<Book> range = bst.getBooksInRange(15, 25);

    EXPECT_EQ(range.size(), 1); // Expected only ISBN 20

    EXPECT_EQ(range[0].Title, "B");
}

TEST(BookBSTTest, InorderIsSorted) {

    BookBST bst;

    bst.addBook(5, "X", "A", 7);

    bst.addBook(2, "Y", "B", 8);

    bst.addBook(8, "Z", "C", 9);

    vecto r<int> inorder = bst.getInorderISBNs();

    EXPECT_EQ(inorder, (vectr<int>{2,5,8})); // Expected sorted ISBNs
}

TEST(BookBSTTest, TotalBooksCountAndHeight) {

    BookBST bst;

    bst.addBook(1, "B1", "Auth1", 15);

    bst.addBook(2, "B2", "Auth2", 25);

    bst.addBook(3, "B3", "Auth3", 35);

    EXPECT_EQ(bst.getTotalBooks(), 3); // Expected count 3

    EXPECT_EQ(bst.getHeight(), 2); // Expected height
}

```

Lab Task 2: Binary Search Tree Compression and Reconstruction

You are part of a development team working on a **distributed storage system**.

In this system, data structures like Binary Search Trees (BSTs) need to be **transmitted across a network** (from one machine to another) or **stored on disk** for later retrieval.

Transmitting the entire pointer-based BST directly is inefficient and unreliable because:

- It contains memory addresses that have no meaning on another machine.
- It consumes large amounts of space.
- It is hard to reconstruct exactly as it was.

To make this possible, you must design an **encoding (compression) and decoding (reconstruction)** scheme for the BST — one that can:

- Convert a BST into a compact, serialized sequence (e.g., a string or list).
- Reconstruct the exact same BST structure from that sequence.

This is sometimes called **BST serialization** and **deserialization**, but here we emphasize the “**compression**” aspect — optimizing how the tree is represented.

Data Model

Each BST node stores:

Key (int) → The BST ordering key
Value (string) → Associated data (e.g., filename, user info, etc.)
Left, Right Pointers → Child nodes

BST Compression (Serialization)

You need to convert the BST into a **compact linear form** (array) that can be transmitted or stored.

Requirements:

- Use **Preorder Traversal (Root → Left → Right)** for serialization.
- Output only the **key** values (and optionally value fields).
- Do **not** include explicit **NULL** markers unless necessary.
- Optimize for space — the goal is “compression,” not just any serialization.

BST Reconstruction (Deserialization)

Given the compressed sequence, reconstruct the **identical BST**.

Rules:

- The order of insertion must produce the original BST structure.
- The reconstruction must work for both balanced and skewed trees.
- Time complexity target: **O(n)** (use preorder + BST constraints, not naive re-insertion $O(n^2)$).

Verification Function

To ensure correctness, provide a function that verifies that:

- The reconstructed BST has the same in-order traversal as the original.
- Both trees have the same structure and values.

You are also expected to create the basic insertion, deletion, traversal functions that are integral to BST implementations

Test Cases

```
#include "gtest/gtest.h"

#include "BST.h"

#include <string>
```

```
#include <sstream>

#include <algorithm>

// Utility: Compare two BSTs for structural + key equality

bool areIdentical(BSTNode* a, BSTNode* b) {

    if (!a && !b) return true;

    if (!a || !b) return false;

    return (a->key == b->key) &&

           areIdentical(a->left, b->left) &&

           areIdentical(a->right, b->right);

}

TEST(BSTCompressionTest, BasicInsertionAndTraversal) {

    BST tree;

    tree.insert(50, "A");

    tree.insert(30, "B");

    tree.insert(70, "C");

    tree.insert(20, "D");

    tree.insert(40, "E");

    tree.insert(60, "F");

    tree.insert(80, "G");
```

```

std::ostream out;

inorderTraversal(tree.root, out);

std::string expected = "20 30 40 50 60 70 80 ";

EXPECT_EQ(out.str(), expected);
}

```

```

TEST(BSTCompressionTest, CompressionProducesCorrectPreorder) {

    BST tree;

    int keys[] = {50, 30, 20, 40, 70, 60, 80};

    const int n = sizeof(keys)/sizeof(keys[0]);

    for (int i = 0; i < n; i++)

        tree.insert(keys[i], "data");

    std::string compressed = tree.compressBST();

    // Expected preorder: 50 30 20 40 70 60 80

    EXPECT_EQ(compressed, "50 30 20 40 70 60 80 ");
}

```

```

TEST(BSTCompressionTest, ReconstructionFromPreorder) {

    BST original;

    int keys[] = {50, 30, 20, 40, 70, 60, 80};

```



```

const int n = sizeof(keys)/sizeof(keys[0]);

for (int i = 0; i < n; i++)

    original.insert(keys[i], "x");

// Compress → Reconstruct

std::string serialized = original.compressBST();

BST reconstructed = BST::reconstructBST(serialized);

EXPECT_TRUE(areIdentical(original.root, reconstructed.root));

// Also verify in-order traversal

std::ostringstream a, b;

inorderTraversal(original.root, a);

inorderTraversal(reconstructed.root, b);

EXPECT_EQ(a.str(), b.str());
}

```

```

TEST(BSTCompressionTest, ReconstructionSkewedRight) {

    BST t1;

    int keys[] = {10, 20, 30, 40, 50};

    const int n = sizeof(keys)/sizeof(keys[0]);

    for (int i = 0; i < n; i++)

```

```

        t1.insert(keys[i], "y");

    std::string seq = t1.compressBST();

    BST t2 = BST::reconstructBST(seq);

    EXPECT_TRUE(areIdentical(t1.root, t2.root));

    std::ostream out;

    inorderTraversal(t2.root, out);

    EXPECT_EQ(out.str(), "10 20 30 40 50 ");
}

TEST(BSTCompressionTest, ReconstructionSkewedLeft) {

    BST t1;

    int keys[] = {50, 40, 30, 20, 10};

    const int n = sizeof(keys)/sizeof(keys[0]);

    for (int i = 0; i < n; i++)

        t1.insert(keys[i], "z");

    std::string seq = t1.compressBST();

    BST t2 = BST::reconstructBST(seq);

    EXPECT_TRUE(areIdentical(t1.root, t2.root));
}

```

```
std::ostream out;

inorderTraversal(t2.root, out);

EXPECT_EQ(out.str(), "10 20 30 40 50 ");

}
```

```
TEST(BSTCompressionTest, EmptyTreeSerialization) {

    BST tree;

    std::string result = tree.compressBST();

    EXPECT_EQ(result, "");

}
```

```
TEST(BSTCompressionTest, SingleNodeReconstruction) {

    BST t1;

    t1.insert(100, "root");

    std::string seq = t1.compressBST();

    BST t2 = BST::reconstructBST(seq);

    EXPECT_TRUE(areIdentical(t1.root, t2.root));

}
```

```
std::ostringstream out;

inorderTraversal(t2.root, out);

EXPECT_EQ(out.str(), "100 ");

}
```

```
TEST(BSTCompressionTest, RandomTreeIntegrity) {

    BST t1;

    int keys[] = {40, 10, 30, 60, 50, 70, 20};

    const int n = sizeof(keys)/sizeof(keys[0]);

    for (int i = 0; i < n; i++)

        t1.insert(keys[i], "data");

    std::string seq = t1.compressBST();

    BST t2 = BST::reconstructBST(seq);

    EXPECT_TRUE(areIdentical(t1.root, t2.root));

    std::ostringstream a, b;

    inorderTraversal(t1.root, a);

    inorderTraversal(t2.root, b);

    EXPECT_EQ(a.str(), b.str());

}
```

```
TEST(BSTCompressionTest, CorruptedInputHandling) {

    std::string corrupted = "50 30 x 20"; // invalid token

    BST reconstructed = BST::reconstructBST(corrupted);

    // Depending on student implementation, tree may be empty or partial.

    EXPECT_TRUE(reconstructed.root == nullptr || reconstructed.root->key
== 50);
}
```